SESIÓN N° 6

Tema: Base de Datos — Fundamentos de SQL y MySQL

I. Objetivo de la Guía

Diseñar, crear y gestionar bases de datos relacionales en MySQL mediante la aplicación de comandos SQL para estructurar la información del sistema y garantizar la integridad referencial.

II. Marco Teórico Breve

1. Bases de Datos Relacionales y MySQL

- Una base de datos relacional organiza los datos en tablas compuestas por filas (registros) y columnas (campos), las cuales se relacionan entre sí mediante claves primarias (PRIMARY KEY) y claves foráneas (FOREIGN KEY). MySQL es uno de los sistemas de gestión de bases de datos relacionales (RDBMS) más populares.
- Piensa en una base de datos relacional como un archivador de oficina con carpetas (tablas): cada carpeta tiene fichas (filas) con casillas (columnas). La PRIMARY KEY es como el número de ficha único que nunca se repite; la FOREIGN KEY es una nota en una ficha que dice "ver la ficha N° X en la carpeta Y".

2. Comandos Esenciales de SQL

- ☆ CREATE DATABASE / CREATE TABLE: Crean la base de datos y las tablas (estructura).
- ☆ INSERT INTO: Agrega una o más filas nuevas a una tabla.
- ☆ SELECT: Consulta (lee) datos de una o varias tablas.
- ☆ UPDATE: Modifica valores de filas existentes (siempre con WHERE).
- ☆ DELETE: Elimina filas (siempre con WHERE para no borrar todo).
- ☆ JOIN: Une datos de dos o más tablas relacionadas en una sola consulta.

Errores Comunes al Empezar

ERROR TÍPICO	CÓMO EVITARLO
Ejecutar DELETE FROM tabla; o UPDATE sin cláusula WHERE.	Sin WHERE se afectan TODAS las filas. Revisa siempre esa cláusula antes de ejecutar.
Insertar en una FOREIGN KEY un valor que no existe en la tabla relacionada.	El valor de una clave foránea debe existir primero como clave primaria en la tabla a la que apunta.
Olvidar las comillas en valores de texto: WHERE nombre = Carlos.	Los valores de texto siempre van entre comillas simples: WHERE nombre = 'Carlos'.

III. Actividades Prácticas Paso a Paso

PARTE 1

Desarrollo del Trabajo 1 (Creación del Esquema Relacional)

Crea la base de datos gamestore_db y las tres tablas del sistema (estudiantes, recompensas, historial_canjes) con sus claves primarias y foráneas para garantizar la integridad referencial.

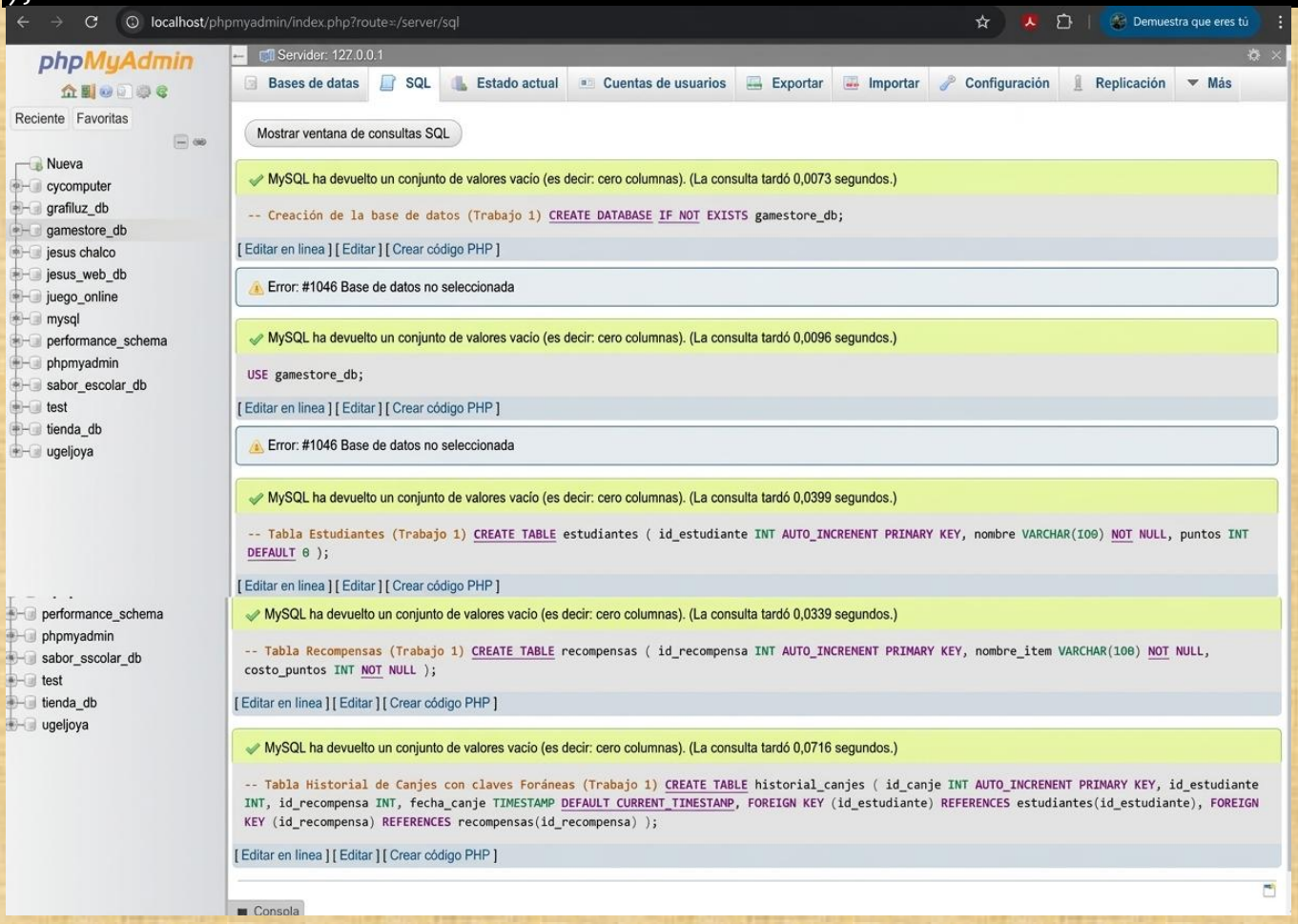
Código SQL completo — Creación del esquema:

```sql
-- Creación de la base de datos (Trabajo 1)
CREATE DATABASE IF NOT EXISTS gamestore_db;
USE gamestore_db;
```

```sql
-- Tabla Estudiantes (Trabajo 1)
CREATE TABLE estudiantes (
    id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    puntos INT DEFAULT 0
);

-- Tabla Recompensas (Trabajo 1)
CREATE TABLE recompensas (
    id_recompensa INT AUTO_INCREMENT PRIMARY KEY,
    nombre_item VARCHAR(100) NOT NULL,
    costo_puntos INT NOT NULL
);

-- Tabla Historial de Canjes con Llaves Foráneas (Trabajo 1)
CREATE TABLE historial_canjes (
    id_canje INT AUTO_INCREMENT PRIMARY KEY,
    id_estudiante INT,
    id_recompensa INT,
    fecha_canje TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante),
    FOREIGN KEY (id_recompensa) REFERENCES recompensas(id_recompensa)
);
```



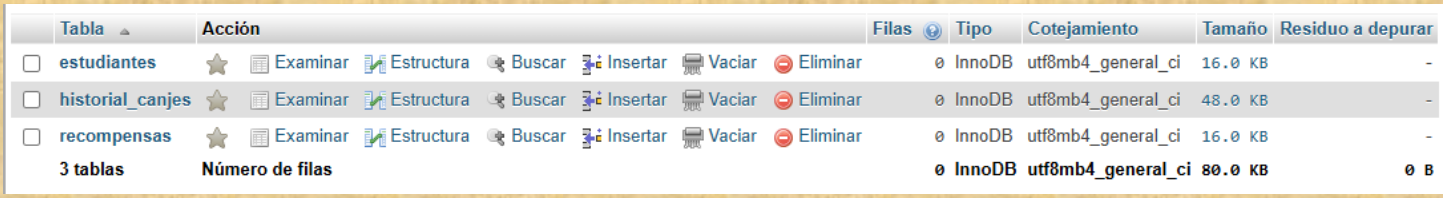


	Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐	estudiantes	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	16.0 KB	-
☐	historial_canjes	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐	recompensas	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	16.0 KB	-
	3 tablas	Número de filas	0	InnoDB	utf8mb4_general_ci	80.0 KB	0 B

Explicación línea por línea (puntos clave):

CREATE DATABASE IF NOT EXISTS gamestore_db;
Crea la base de datos gamestore_db, solo si todavía no existe (evita error si ya la creaste antes).

USE gamestore_db;
Selecciona gamestore_db como la base de datos activa: todas las instrucciones siguientes se ejecutarán dentro de ella.

CREATE TABLE estudiantes (
Inicia la creación de la tabla estudiantes, abriendo el paréntesis con la lista de columnas.

id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
Columna id_estudiante: número entero que se autoincrementa solo (1, 2, 3...) y es la PRIMARY KEY (identificador único) de la tabla.

nombre VARCHAR(100) NOT NULL,
Columna nombre: texto de hasta 100 caracteres, obligatorio (NOT NULL: no puede quedar vacío).

puntos INT DEFAULT 0
Columna puntos: número entero; si no se especifica un valor al insertar, toma 0 por defecto.

);
Cierra la definición de columnas y termina la sentencia CREATE TABLE.

CREATE TABLE recompensas (
Inicia la creación de la tabla recompensas.

id_recompensa INT AUTO_INCREMENT PRIMARY KEY,
Columna id_recompensa: PRIMARY KEY autoincremental, igual que en estudiantes.

nombre_item VARCHAR(100) NOT NULL,
Columna nombre_item: texto de hasta 100 caracteres, obligatorio.

costo_puntos INT NOT NULL
Columna costo_puntos: número entero, obligatorio (sin valor por defecto).

);
Cierra la definición de columnas de recompensas.

CREATE TABLE historial_canjes (
Inicia la creación de la tabla historial_canjes.

id_canje INT AUTO_INCREMENT PRIMARY KEY,
Columna id_canje: PRIMARY KEY autoincremental de esta tabla.

id_estudiante INT,
Columna id_estudiante: guardará el id de QUÉ estudiante hizo el canje (todavía sin la restricción de llave foránea; eso viene más abajo).

id_recompensa INT,
Columna id_recompensa: guardará el id de QUÉ recompensa se canjeó.

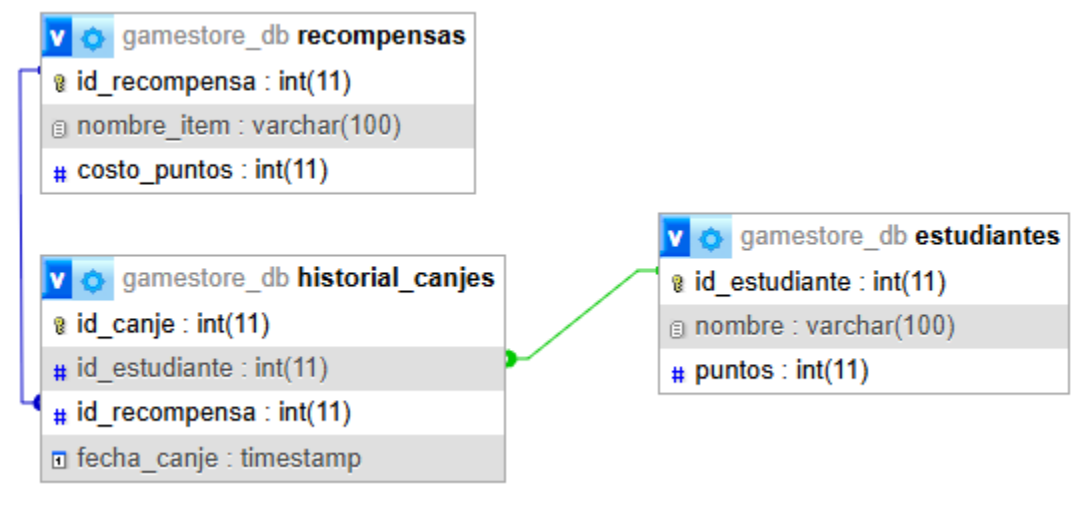
fecha_canje TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
Columna fecha_canje: marca de tiempo que se llena sola con la fecha/hora actual si no se especifica otra.

FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante),
Restricción FOREIGN KEY: obliga a que todo valor de id_estudiante exista realmente como id_estudiante en la tabla estudiantes.

FOREIGN KEY (id_recompensa) REFERENCES recompensas(id_recompensa)
Restricción FOREIGN KEY: obliga a que todo valor de id_recompensa exista realmente como id_recompensa en la tabla recompensas.

);
Cierra la definición de columnas y restricciones, terminando la sentencia CREATE TABLE de historial_canjes.

Diagrama de relaciones:



PARTE 2

Desarrollo del Trabajo 2 (Manipulación de Datos y Consultas)

Inserta datos de prueba, actualiza puntajes, registra un canje y realiza una consulta con JOIN para ver el historial detallado relacionando las tres tablas.

Código SQL completo — DML y JOIN:

```sql
-- Inserción de registros iniciales (Trabajo 2)
INSERT INTO estudiantes (nombre, puntos) VALUES ('Carlos Mendoza', 500), ('Ana Torres', 350);
INSERT INTO recompensas (nombre_item, costo_puntos) VALUES ('Skin Guerrero Cyber', 150),
('Pase de Prórroga', 300);

-- Consulta general de estudiantes
SELECT * FROM estudiantes;

-- Actualización de puntajes (Trabajo 2)
UPDATE estudiantes SET puntos = 600 WHERE id_estudiante = 1;

-- Registro de un canje relacionando las tablas (Trabajo 2)
INSERT INTO historial_canjes (id_estudiante, id_recompensa) VALUES (1, 1);

-- Consulta avanzada con JOIN para ver el historial detallado de canjes (Trabajo 2)
SELECT h.id_canje, e.nombre AS estudiante, r.nombre_item, r.costo_puntos, h.fecha_canje
FROM historial_canjes h
JOIN estudiantes e ON h.id_estudiante = e.id_estudiante
JOIN recompensas r ON h.id_recompensa = r.id_recompensa;
```


Servidor: 127.0.0.1 » Base de datos: gamestore_db » Tabla: estudiantes
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento
Mostrar ventana de consultas SQL
2 filas insertadas.
La Id de la fila insertada es: 2 (La consulta tardó 0,0086 segundos.)
-- Inserción de registros iniciales (Trabajo 2) INSERT INTO estudiantes (nombre, puntos) VALUES ('Carlos Mendoza', 500), ('Ana Torres', 350);
[Editar en línea] [Editar] [Crear código PHP]
2 filas insertadas.
La Id de la fila insertada es: 2 (La consulta tardó 0,0123 segundos.)
INSERT INTO recompensas (nombre_item, costo_puntos) VALUES ('Skin Guerrero Cyber', 150), ('Pase de Prórroga', 300);
[Editar en línea] [Editar] [Crear código PHP]
Mostrando filas 0 - 1 (total de 2, La consulta tardó 0,0005 segundos.)
-- Consulta general de estudiantes SELECT * FROM estudiantes;
[Editar en línea] [Editar] [Crear código PHP]
Mostrar todo | Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna
1 fila afectada. (La consulta tardó 0,0007 segundos.)
-- Actualización de puntajes (Trabajo 2) UPDATE estudiantes SET puntos = 600 WHERE id_estudiante = 1;
[Editar en línea] [Editar] [Crear código PHP]
1 fila insertada.
La Id de la fila insertada es: 1 (La consulta tardó 0,0008 segundos.)
-- Registro de un canje relacionando las tablas (Trabajo 2) INSERT INTO historial_canjes (id_estudiante, id_recompensa) VALUES (1, 1);
[Editar en línea] [Editar] [Crear código PHP]
La selección actual no contiene una columna única. La edición de la grilla y los enlaces de copiado, eliminación y edición no están disponibles.
Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0008 segundos.)
-- Consulta avanzada con JOIN para ver el historial detallado de canjes (Trabajo 2) SELECT h.id_canje, e.nombre AS estudiante, r.nombre_item, r.costo_puntos, h.fecha_canje FROM historial_canjes h JOIN estudiantes e ON h.id_estudiante = e.id_estudiante JOIN recompensas r ON h.id_recompensa = r.id_recompensa;
[Editar en línea] [Editar] [Crear código PHP]
id_estudiante nombre puntos
Editar Copiar Borrar 1 Carlos Mendoza 500
Editar Copiar Borrar 2 Ana Torres 350
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
id_canje estudiante nombre_item costo_puntos fecha_canje
1 Carlos Mendoza Skin Guerrero Cyber 150 2026-11-03 10:50:14

Explicación línea por línea (puntos clave):

INSERT INTO estudiantes (nombre, puntos) VALUES ('Carlos Mendoza', 500), ('Ana Torres', 350);
Inserta dos estudiantes nuevos en la tabla estudiantes, en una sola sentencia (dos tuplas de valores separadas por coma).
INSERT INTO recompensas (nombre_item, costo_puntos) VALUES ('Skin Guerrero Cyber', 150), ('Pase de Prórroga', 300);
Inserta dos recompensas nuevas en la tabla recompensas, también en una sola sentencia.
SELECT * FROM estudiantes;
SELECT * FROM estudiantes: pide TODAS las columnas (*) de TODAS las filas de la tabla estudiantes.
UPDATE estudiantes SET puntos = 600 WHERE id_estudiante = 1;
UPDATE cambia el valor de puntos a 600, pero SOLO en la fila donde id_estudiante sea igual a 1 (gracias al WHERE); sin el WHERE, se actualizarían TODAS las filas.
INSERT INTO historial_canjes (id_estudiante, id_recompensa) VALUES (1, 1);
Inserta una fila en historial_canjes indicando que el estudiante con id 1 canjeó la recompensa con id 1 (fecha_canje se llena sola).
SELECT h.id_canje, e.nombre AS estudiante, r.nombre_item, r.costo_puntos, h.fecha_canje
Selecciona columnas específicas de tres tablas distintas a la vez, usando alias (h, e, r) para abreviar sus nombres.
FROM historial_canjes h
Indica que la tabla principal de la consulta es historial_canjes, con el alias h.
JOIN estudiantes e ON h.id_estudiante = e.id_estudiante
Une (JOIN) historial_canjes con estudiantes, emparejando cada canje con el estudiante cuyo id coincide.
JOIN recompensas r ON h.id_recompensa = r.id_recompensa;
Une (JOIN) el resultado anterior con recompensas, emparejando cada canje con la recompensa cuyo id coincide.

Resultado esperado del JOIN (ejemplo):

id_canje	estudiante	nombre_item	costo_puntos	fecha_canje
1	Carlos Mendoza	Skin Guerrero Cyber	150	2026-11-03 10:50:14
NULL	Ana Torres	*NULL*	*NULL*	*NULL*

Resumen de Conceptos Clave

- ☆ **PRIMARY KEY**: Columna que identifica de forma única cada fila de una tabla (no se repite, no es NULL).
- ☆ **FOREIGN KEY**: Columna que conecta una tabla con la PRIMARY KEY de otra; garantiza integridad referencial.
- ☆ **INSERT INTO**: Agrega filas nuevas a una tabla.
- ☆ **SELECT**: Consulta (lee) datos. SELECT * devuelve todas las columnas.
- ☆ **UPDATE ... WHERE**: Modifica valores solo en las filas que cumplen la condición.
- ☆ **JOIN ... ON**: Une filas de varias tablas emparejándolas por la relación de claves (PK–FK).
- ☆ **AUTO_INCREMENT**: MySQL genera automáticamente el siguiente número entero para la PRIMARY KEY al insertar.